

Research Report

Authors : Youssef Barhoumi , Zeineb Mekki

Part 1 — The Modeling Core

1. Data Augmentation

Artificially expanding training data via transforms (flips, rotations, color jitter) to reduce overfitting by preventing the model from memorizing specific pixels.

```
1 transform = transforms.Compose([
2     transforms.RandomHorizontalFlip(),
3     transforms.RandomRotation(15),
4     transforms.ColorJitter(brightness=0.3, contrast=0.3),
5     transforms.ToTensor()
6 ])
```

Reference: Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on Image Data Augmentation for Deep Learning. *Journal of Big Data*, 6(1), 60.

2. PyTorch & Dynamic Computation Graphs

PyTorch builds the computation graph on-the-fly during the forward pass, making standard Python control flow work naturally and debugging straightforward.

```
1 x = torch.randn(4, 3, 224, 224, requires_grad=True)
2 model = nn.Sequential(nn.Flatten(), nn.Linear(3*224*224, 10))
3 loss = nn.CrossEntropyLoss()(model(x), torch.randint(0, 10, (4,)))
4 loss.backward()
```

Reference: Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*, 32.

3. CNNs — Convolutional & Pooling Layers

Conv layers slide a learnable filter over the input to detect local patterns; MaxPool down-samples by keeping the max value in each window, achieving spatial invariance.

```
1 cnn = nn.Sequential(
2     nn.Conv2d(3, 32, kernel_size=3, padding=1),
3     nn.ReLU(),
4     nn.MaxPool2d(2, 2)
5 )
```

Reference: LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.

4. Transfer Learning — Freezing & Fine-Tuning

Reuse a pretrained backbone (e.g. ResNet), freeze its weights so only a new head trains first, then optionally unfreeze layers at a low learning rate to adapt features to the new task.

```
1 model = models.resnet50(weights="IMAGENET1K_V2")
2 for param in model.parameters():
3     param.requires_grad = False
4 model.fc = nn.Linear(2048, 5)
5 for param in model.layer4.parameters():
6     param.requires_grad = True
```

Reference: Pan, S. J., & Yang, Q. (2010). A Survey on Transfer Learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10), 1345–1359.

5. Adam Optimizer

Combines momentum (m_t) and RMSProp (v_t) to give each parameter its own adaptive learning rate — robust, fast-converging, and the default choice for most deep learning tasks.

```
1 optimizer = optim.Adam(model.parameters(), lr=3e-4, betas=(0.9,
    0.999))
2 loss = criterion(model(x), y)
3 loss.backward()
4 optimizer.step()
```

Reference: Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015*.

Part 2 — User Experience & Trust

6. XAI & Grad-CAM

Grad-CAM flows the class score gradient back to the last conv layer, weights its feature maps, and produces a heatmap showing which image regions drove the prediction.

```
1 cam = GradCAM(model=model, target_layers=[model.layer4[-1]])
2 grayscale_cam = cam(input_tensor=img_tensor)[0]
3 visualization = show_cam_on_image(rgb_img, grayscale_cam, use_rgb
    =True)
```

Reference: Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *ICCV 2017*, 618–626.

7. Streamlit

Re-runs a Python script top-to-bottom on each interaction, turning it into a browser UI with no HTML/CSS/JS needed. Ideal for exposing model inference as a demo.

```
1 st.title("Skin Lesion Classifier")
2 uploaded = st.file_uploader("Upload an image", type=["jpg", "png"]
3   )
4 if uploaded:
5     img = Image.open(uploaded)
6     st.image(img)
7     st.success(f"Prediction: {model_predict(img)}")
```

Reference: Streamlit Inc. (2020). Streamlit: The fastest way to build and share data apps. <https://streamlit.io>